

GitNexus

Proyecto de Plataforma de Métricas y Calidad para GitHub

Project Charter (Acta de Constitución del Proyecto)

Juan Sebastian Perafan Bohorquez,
Johan Alejandro Bernal,
Carlos Alberto Caro Quezada,
Cristian Bayona Alzate,
Daniel Castillo Mora,
Santiago Salgado

Fecha de Generación:

20/05/2026

Tabla de contenido

PROJECT CHARTER - GitNEXUS

1. INFORMACIÓN GENERAL DEL PROYECTO	3
2. PROPÓSITO / JUSTIFICACIÓN	3
3. DESCRIPCIÓN DEL PROYECTO	3
4. OBJETIVOS	4
5. ALCANCE	4
6. ENTREGABLES PRINCIPALES	5
7. REQUERIMIENTOS DE ALTO NIVEL	5
8. SUPUESTOS Y RESTRICCIONES	6
9. RIESGOS INICIALES	6
10. CRONOGRAMA DE ALTO NIVEL	7
11. EQUIPO DEL PROYECTO	7
12. ARQUITECTURA INICIAL PROPUESTA	8
13. INTERESADOS	8
14. CRITERIOS DE ÉXITO	8
15. DEPENDENCIAS	9
16. APROBACIONES Y FIRMAS	

Lista de Tablas

Tabla 1. Matriz de Riesgos Iniciales del Proyecto.	6
Tabla 2. Cronograma de Alto Nivel e Hitos del Proyecto.	7
Tabla 3. Estructura del Equipo del Proyecto.	8

1. INFORMACIÓN GENERAL DEL PROYECTO

Nombre del proyecto: GitNexus
Código del proyecto: GIQA-2026
Versión: 1.0
Fecha de inicio: 15/05/2026
Fecha de Finalización: 07/10/2026
Duración Estimada: 5 meses

El proyecto busca diseñar e implementar una plataforma centralizada para recopilar, analizar y visualizar métricas provenientes de GitHub con el objetivo de evaluar calidad de código, productividad, actividad de desarrolladores, estado de repositorios y cumplimiento de estándares de desarrollo.

2. PROPÓSITO / JUSTIFICACIÓN

En el desarrollo de software es muy utilizada la plataforma de github, los proyectos dependen de múltiples repositorios y equipos de desarrollo distribuidos. Sin embargo, muchas veces no existe una visibilidad clara sobre la calidad del código, productividad, mantenibilidad o riesgos técnicos.

Este proyecto nace con el propósito de:

- Centralizar información técnica de GitHub.
- Detectar riesgos de calidad y deuda técnica.
- Facilitar la toma de decisiones basada en métricas.
- Medir desempeño técnico de equipos y repositorios.
- Verificar el trabajo de los contribuyentes en un equipo de desarrollo.

3. DESCRIPCIÓN DEL PROYECTO

Se desarrollará una plataforma web capaz de conectarse a GitHub mediante APIs oficiales para extraer, procesar y visualizar metadatos clave de desarrollo para recolectar información relacionada con:

- **Gestión de Repositorios:** Análisis de volumen de código, obsolescencia de ramas y frecuencia de integración.
- **Evaluación de Commits:** Verificación de buenas prácticas en la mensajería, granularidad del código e impacto semántico de las modificaciones.
- **Control de Integrantes:** Mapeo de contribuciones, tasa de aprobación de revisiones (Code Review) y participación equitativa en el flujo de trabajo.

4. OBJETIVOS

Objetivo General:

Desarrollar una Plataforma integral para analizar y evaluar métricas de GitHub que extraiga y procese métricas de rendimiento relacionadas con calidad de código, desempeño de equipos y estado de repositorios con el fin de llevar un seguimiento con respecto al desarrollo de un proyecto.

Objetivos Específicos:

- Desarrollar un Backend que gestione request al API REST de GitHub.
- Automatizar tareas de cálculo y reporte de KPIs.
- Visualizar datos para análisis y toma de decisiones.
- Implementar infraestructura como código (IaC).

5. ALCANCE

5.1. Dentro del Alcance (In-Scope)

- Implementar una integración segura con GitHub mediante APIs, OAuth/Webhooks y mecanismos de sincronización automática de eventos y repositorios.
- Desarrollar un pipeline automatizado para la recolección, procesamiento y correlación de métricas técnicas relacionadas con actividad de desarrollo, calidad de código y estabilidad de repositorios.
- Construir dashboards interactivos y visualizaciones analíticas con filtros dinámicos, indicadores de rendimiento y métricas para el monitoreo técnico.
- Implementar un sistema de reportería capaz de generar exportaciones en múltiples formatos (PDF, XLSX) para auditoría y análisis ejecutivo.
- Incorporar mecanismos de análisis de calidad y deuda técnica para detectar problemas estructurales, incumplimiento de estándares y cuellos de botella en revisiones de código.
- Proporcionar herramientas de monitoreo de actividad de colaboradores para evaluar participación, flujo de trabajo y balance operativo dentro de los equipos de desarrollo.

5.2. Fuera del Alcance (Out-of-Scope)

- Aplicación móvil nativa.
- Gestión de roles y permisos complejos.
- Integración con plataformas diferentes a GitHub.

6. ENTREGABLES PRINCIPALES

Documentación y Especificación Técnicas: Paquete documental consolidado que incluye la Especificación de Requerimientos de Software (ERS), el Documento de Arquitectura de Software (DAS) con diagramas del flujo de trabajo y el Manual de Usuario.

Componentes de Backend e Integración con GitHub: Código fuente contenerizado del backend que implementa la API de integración segura con GitHub, junto con el esquema y despliegue de una Base de Datos centralizada optimizada para el almacenamiento de series de tiempo operativas.

Módulos de Seguridad y Core de Procesamiento Analítico: Módulo funcional de Control de Acceso Basado en Roles y un Motor Analítico automatizado encargado de auditar la calidad del código, procesar la metadata de Git y computar métricas de rendimiento técnico.

Capa de Presentación, Reportería y Despliegue en Producción: Interfaz web interactiva (Dashboard) para la visualización de KPIs en tiempo real, un Subsistema de reportes ejecutivos exportables (PDF, XLSX) y la infraestructura de la plataforma completamente desplegada y operativa en entorno de producción Cloud.

7. REQUERIMIENTOS DE ALTO NIVEL

Requerimientos Funcionales:

- El sistema debe permitir autenticación segura y gestionar accesos con permisos controlados.
- El sistema debe sincronizar automáticamente repositorios, ramas y metadatos desde GitHub mediante procesos programados y eventos asíncronos.
- El sistema debe calcular y visualizar métricas de calidad de código y desempeño técnico, tiempos de revisión, cumplimiento de estándares y ciclos de integración
- La plataforma debe ofrecer filtros dinámicos y segmentación de métricas por usuario, equipo, repositorio y periodos de tiempo.
- El sistema debe permitir la generación y exportación de reportes ejecutivos en formatos PDF y EXCEL para análisis y auditoría.

Requerimientos no Funcionales:

- **Seguridad:** Cifrado de credenciales de GitHub y tokens de acceso en reposo mediante el algoritmo AES-256.
- **Rendimiento:** El tiempo de carga inicial de las métricas agregadas del dashboard principal no debe superar los 2.5 segundos.
- **Disponibilidad:** La plataforma debe garantizar una disponibilidad operativa del 99.5% (SLA) en entornos de producción.

8. SUPUESTOS Y RESTRICCIONES

Supuestos:

- Disponibilidad de la API de GitHub.
- Acceso a repositorios públicos para pruebas.
- Se asume disponibilidad continua del equipo de desarrollo.
- Se asume disponibilidad y acceso continuo a las herramientas tecnológicas necesarias.

Restricciones:

- Tiempo máximo de finalización: 5 meses.
- Desarrollo de Backend en Python (FastAPI) o Java (Spring Boot).
- Uso de API REST o GraphQL de GitHub.
- Base de datos en PostgreSQL.
- Despliegue en Docker y Terraform.

9. RIESGOS INICIALES

La siguiente matriz identifica los principales riesgos técnicos y operativos del proyecto, evaluando su nivel de impacto y probabilidad de ocurrencia, junto con las estrategias de mitigación definidas para reducir sus efectos sobre el desarrollo, la seguridad y la estabilidad de la solución.

Tabla 1. Matriz de Riesgos Iniciales del Proyecto.

Riesgo Identificado	Impacto / Probabilidad	Estrategia de Mitigación
Exceder el límite de peticiones impuesto por la API de GitHub (Rate Limiting).	Alto / Media	Implementar una arquitectura dirigida por eventos mediante la suscripción a Webhooks nativos de GitHub, eliminando la necesidad de realizar consultas periódicas por tiempo.
Resistencia del equipo de desarrollo al sentirse "micro gestionado" por las métricas de commits.	Medio / Alta	Establecer un enfoque de métricas orientado a la salud del proceso y no al castigo individual. Enfocar los KPIs en la optimización colectiva.
Brechas de seguridad por filtración de Tokens de Acceso Personal (PAT).	Crítico / Baja	Utilizar integraciones tipo GitHub Apps con permisos mínimos necesarios y rotación automatizada de llaves secretas.

10. CRONOGRAMA DE ALTO NIVEL / HITOS

La siguiente tabla presenta los principales hitos del proyecto y su planificación temporal estimada, permitiendo visualizar las etapas clave del desarrollo, integración, automatización y despliegue de la solución basada en la API de GitHub.

Tabla 2. Cronograma de Alto Nivel e Hitos del Proyecto.

Hito	Descripción del Milestone	Fecha Estimada
Mes 1	Aprobación formal del Project Charter y Diseño de Arquitectura.	15/06/2026
Mes 2	Desarrollo del Core del Backend e Integración con API de GitHub.	15/07/2026
Mes 3	Procesamiento de Datos y Cálculo de KPIs.	15/08/2026
Mes 4	Desarrollo Frontend y conexión con los endpoints del Backend.	15/09/2026
Mes 5	Automatización de tareas y ajustes finales. Despliegue, documentación y presentación final.	07/10/2026

11. EQUIPO DEL PROYECTO

La siguiente tabla describe los roles principales involucrados en el proyecto y sus responsabilidades generales dentro del desarrollo, integración, despliegue y validación de la plataforma basada en la API de GitHub.

Tabla 3. Estructura del Equipo del Proyecto

Rol	Responsabilidad
Project Manager	Planificación, seguimiento y coordinación
Desarrollador Backend	Consumo de API REST y lógica del sistema.
Desarrollador Frontend	Interfaz y visualización de KPIs
DevOps	Docker, despliegue e infraestructura
QA Tester	Pruebas funcionales y validación.

12. ARQUITECTURA INICIAL PROPUESTA

Backend:

- Python
- FastAPI
- API REST

Frontend:

- React

Base de Datos:

- PostgreSQL

Control de Versiones:

- Github

Despliegue:

- Docker
- Terraform

13. INTERESADOS (STAKEHOLDERS)

Patrocinador ejecutivo: Grupo Asd, quien se encargará de proveer el financiamiento y evaluación de los resultados.

Líderes técnicos: Serán los encargados de visualizar y monitorear el trabajo de los desarrolladores durante los próximos meses, encargándose de evaluar y retroalimentar el trabajo que estos realicen.

14. CRITERIOS DE ÉXITO

El proyecto será considerado exitoso si cumple los siguientes requisitos:

- El sistema logra procesar e indexar el 100% de los eventos enviados por los Webhooks de GitHub sin pérdida de paquetes informáticos.
- Los dashboards muestran correctamente las métricas en tiempo real.
- La disponibilidad del sistema supera 99% y todas sus acciones, además de gestiones, funcionan correctamente.

15. DEPENDENCIAS

- Dependencia directa de la estabilidad de los servicios globales de GitHub Enterprise/Cloud.
- Aprovisionamiento y administración correcta de credenciales OAuth 2.0, GitHub Apps y tokens de acceso con permisos adecuados sobre los repositorios y organizaciones objetivo.
- Disponibilidad de infraestructura cloud y recursos computacionales necesarios para el procesamiento analítico, almacenamiento persistente y despliegue de los servicios de la plataforma.